9장. 예외처리, 파일 입출력

python™

에러(Error)와 예외(Exception)

에러(Error)

- **구문(syntex) 오류** : 문법에 맞지 않거나 오타가 났을 경우 발생하는 오류
  IDE에서 실행 전에 알 수 있음

예외(Exception)

- **실행(runtime) 오류** : 문법적인 오류는 없지만 실행(run) 될 때 에러가 발생하는 것을 말한다.

  예) 파일을 읽어 사용하려는데 파일이 없는 경우,
  리스트 값을 출력하려는데 리스트 요소가 없는 경우 등..
  에러가 발생되면 프로그램의 동작이 중지 또는 종료된다

예외(Exceptions)

❖ python.org > Tutorial(자습서)

8.2. Exceptions

Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it. Errors detected during execution are called *exceptions* and are not unconditionally fatal: you will soon learn how to handle them in Python programs. Most exceptions are not handled by programs, however, and result in error messages as shown here:

Copy

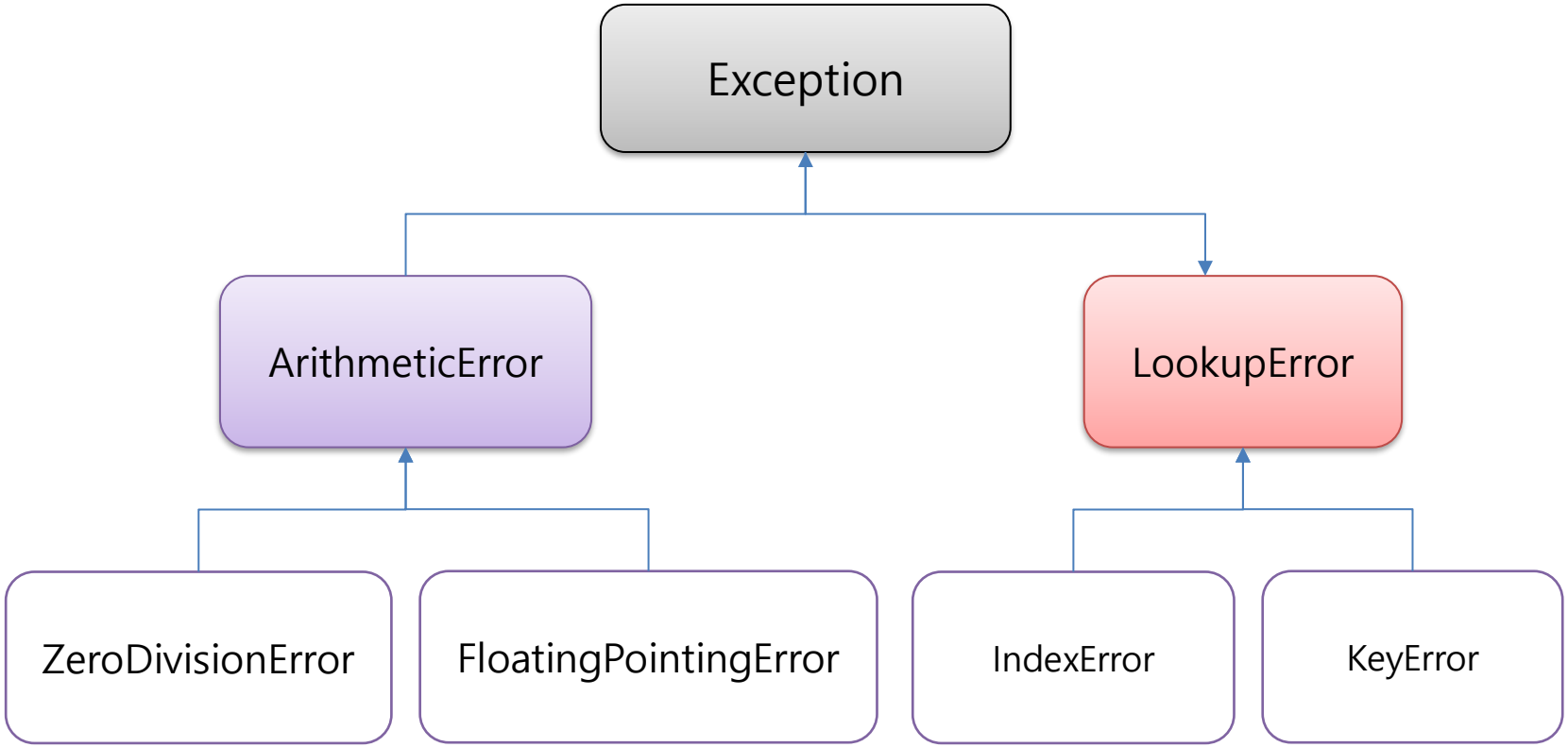
```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    10 * (1/0)
          ~^~
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    4 + spam*3
        ^^^^
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    '2' + 2
    ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

예외(Exceptions)

❖ 여러가지 예외 발생 코드

```
print('2' + 3) # TypeError 발생


# print(4 / 0)  # ZeroDivisionError 발생


# calculation = 10 + number  # NameError 발생
```

Exception 계층도



예외(Exceptions)

❖ **예외 처리 방법** : **try ~ except 구문**

try:

예외가 발생할 가능성이 있는 코드

except 예외 클래스:

예외가 발생했을 경우 실행 코드

예외(Exceptions)

- 예외 처리 코드

```
try:
  print('20' + 20)
except TypeError as e:
  print("자료형 오류가 발생했습니다:", e)
  # print('TypeError:', e)

try:
  print(10 / 0)
except ZeroDivisionError as e:
  print("0으로 나눌 수 없습니다:", e)
  # print('ZeroDivisionError:', e)

try:
  message = "Good Luck!" + friend
except NameError as e:
  print("변수가 정의되지 않았습니다:", e)
  # print('NameError:', e)
```

예외(Exceptions)

- 숫자를 입력할 곳에 문자를 입력하여 예외 발생

```
숫자를 입력하세요: haha
Traceback (most recent call last):
  File "d:\더조은종로메가패스\pyworks\예외처리\exception01.py",
    number = int(input("숫자를 입력하세요: "))
             ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'haha'
```

```
try:
    number = int(input("숫자를 입력하세요: "))
    print(number + 10)
except ValueError as e:
    print("유효한 숫자를 입력하세요:", e)
    # print('ValueError:', e)
```

다중 예외(Exceptions)

- 다중 try ~ except 구문

```
try:
    실행 코드
except 오류
    문제 발생시 실행코드
except 오류
    문제 발생시 실행코드
```

다중 예외(Exceptions)

➢ 다중 try ~ except 구문

```
try:
    data = [20, 10, 30, 50]
    print(data[4])  # IndexError 발생
    print(data[3] / 0)  # ZeroDivisionError 발생
except IndexError:
    print("리스트 인덱스가 범위를 벗어났습니다.")
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
```

예외 처리 미루기

➢ raise 구문

- raise 예외클래스("예외 메시지")
- 예외를 의도적으로 발생시킴

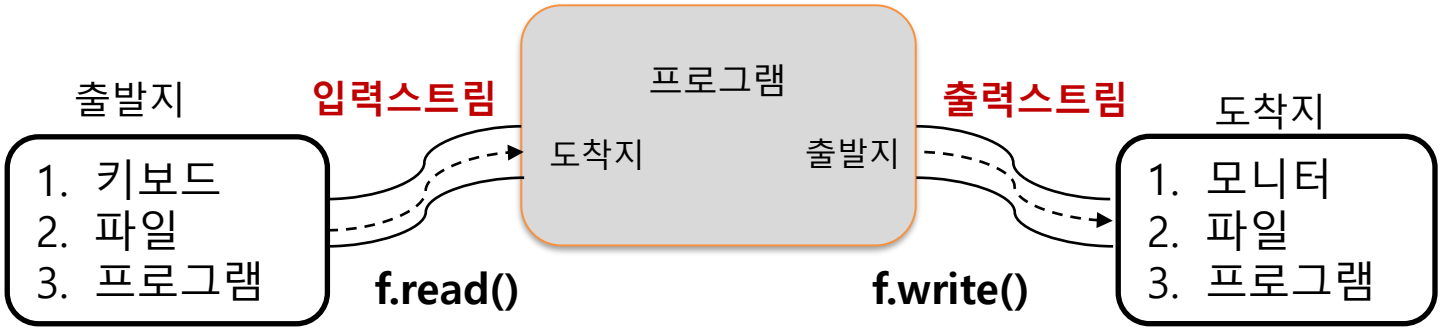
```
def divide(a, b):
    if b == 0:
        raise ValueError("b는 0이 될 수 없습니다.")
    return a / b

try:
    result = divide(10, 0)
    print(result)
except ValueError as e:
    print("값 오류가 발생했습니다:", e)
```

입, 출력 스트림

- **스트림(stream)**

  자료흐름이 물의 흐름과 같다는 뜻이다. 입출력 장치는 매우 다양하기 때문에 프로그램 호환성이 떨어짐

  - 입력 스트림 – 동영상을 재생하기 위해 동영상 파일에서 자료를 읽을때 사용함
  - 출력 스트림 – 사용자가 쓴 글을 파일에 저장할 때는 출력 스트림 사용함

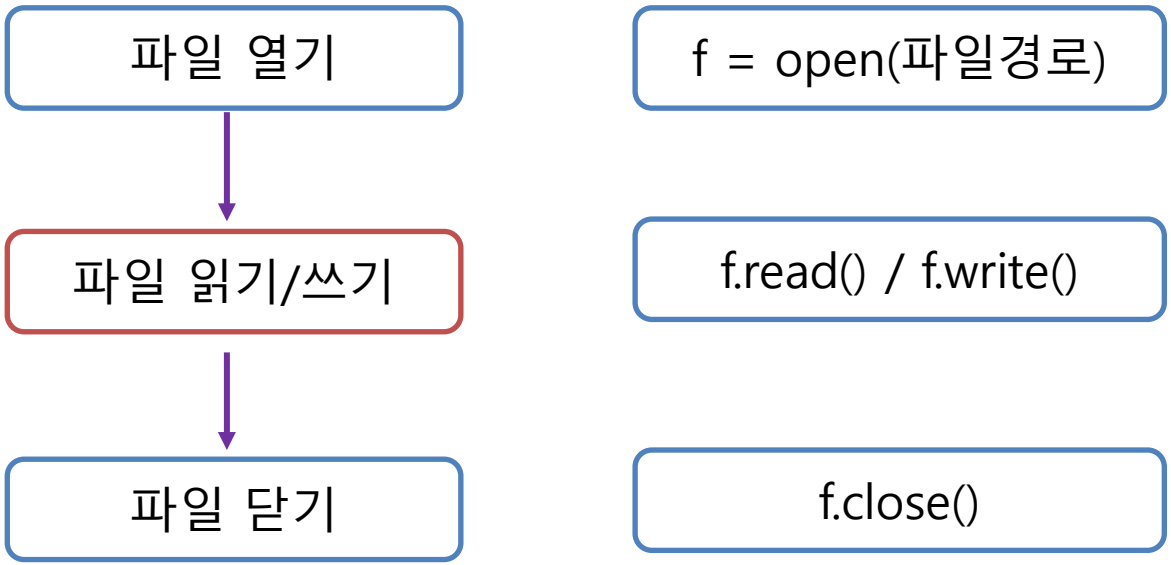


파일 입출력

- **파일 입출력의 필요성**

  프로그램 실행 중에 메모리에 저장된 데이터는 프로그램이 종료되면 사라진다.
  데이터를 프로그램이 종료된 후에도 계속해서 사용하려면 파일에 저장하고
  필요할 때 파일을 읽어서 데이터를 사용할 수 있다.

- **파일 입출력 프로세스**



파일 쓰기

- **파일 관련 주요 메서드**

메서드	모드	기능
open(파일, “w”)	w	파일 열기(쓰기)
open(파일, “r”)	r	파일 열기(읽기)
open(파일, “a”)	a	파일 열기(추가 쓰기)
write()		파일 내용 쓰기
read()		파일 내용 읽기
close()		파일 닫기

파일 쓰기

- **파일 쓰기**

```python
# 파일 열기 - 쓰기 모드
f = open('c:/pyfile/file1.txt', 'w')

# 파일에 문자열 작성
f.write('Hello Python\n')
f.write('즐거운 하루 되세요~\n')
# f.write(30) # TypeError 발생
f.write(str(30) + '\n') # 숫자를 문자열로 변환하여 작성

# 파일 닫기
f.close()
```

file.txt

```
Hello Python
즐거운 하루 되세요~
30
```

파일 읽기

- 파일 읽기

```python
# 파일 열기 - 읽기 모드
try:
    f = open('c:/pyfile/file1.txt', 'r')

    # 파일에 작성된 문자열 읽기
    content = f.read()
    print(content)
except FileNotFoundError as e:
    print("파일을 찾을 수 없습니다:", e)

# 파일 닫기
f.close()
```

파일 쓰기(추가모드)

- 파일 쓰기(추가 모드)

```
# try-except-finally 구문을 사용

# 파일 열기 - 추가 모드
try:
    f = open('c:/pyfile/file1.txt', 'a')

    # 파일에 문자열 작성
    f.write('추가된 내용입니다.\n')
    f.write('Hello World\n')
except FileNotFoundError as e:
        print("파일을 찾을 수 없습니다:", e)

finally:
    if 'f' in locals(): # 파일이 열렸는지 확인
        f.close()
```

with ~ as 구문

- **with ~ as 구문**

with open(파일이름) as 파일 객체:
코드 블럭

- **자동 닫기** — with 블록을 벗어나면 에러 발생 여부와 관계없이 자동으로 close()가 호출됩니다.
- **코드가 간결** — try-finally + close()를 직접 쓸 필요가 없습니다.
- **안전** — close()를 깜빡해도 문제가 없습니다.

with ~ as 구문 예제

- 구구단 파일 쓰기

```python
# 구구단을 파일로 저장하기
with open('c:/pyfile/gugudan.txt', 'w') as f:
    for i in range(2, 10):
        for j in range(1, 10):
            f.write(f"{i} x {j} = {i*j}\n")
        f.write("\n")
# 파일 닫기는 with 블록이 끝나면 자동으로 처리됩니다.

# 파일 열기 - 읽기 모드
try:
    with open('c:/pyfile/gugudan.txt', 'r') as f:
        content = f.read()
        print(content)
except FileNotFoundError as e:
    print("파일을 찾을 수 없습니다:", e)
```

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18

3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
```

with ~ as 구문 예제

- 파일에 리스트 자료 쓰고 읽기

```
# 파일에 리스트 저장하기
carts = ["라면", "달걀", "쌀", "커피"]

# 한글이 깨지는 경우 방지 - > encoding='utf-8' 추가
with open('output/cart.txt', 'w', encoding='utf-8') as f:
    for item in carts:
        f.write(f"{item}\n")

# 파일에서 리스트 불러오기
with open('output/cart.txt', 'r', encoding='utf-8') as f:
    content = f.read()
    print(content) # 라면\n달걀\n쌀\n커피\n

    # 한 줄씩 읽어서 출력
    # for line in f:
    #     print(line.strip()) #줄바꿈 문자 제거
```

영어 타자 연습 프로그램

- **영어 타자 게임**

게임 방법

- 파일 쓰기를 이용하여 word.txt 파일을 생성한다.
- 게임이 시작되면 영어 단어가 화면에 표시된다.
- 사용자는 최대한 빠르고 정확하게 입력해야 한다.
- 바르게 입력했으면 다음 문제로 넘어가고 "통과"를 출력한다.
- 오타가 있으면 '오타! 다시 도전!'이 출력되고 같은 단어가 한 번 더 나온다.
- 타자 게임 시간을 측정한다.

영어 타자 연습 프로그램

- **영어 타자 게임**

```
[타자 게임]준비되면 엔터!

-문제 1
grape
grape
통과!
-문제 2
potato
potata
오타! 다시 도전!
-문제 2
potato
potato
통과!
-문제 3
grape
```

```
-문제 9
tree
tree
통과!
-문제 10
garlic
garlic
통과!
타자 시간 : 34.46초
```

리스트 랜덤 출력

- **word.txt 파일 만들고, 랜덤 추출하기**

```
import random

# 구구단을 파일로 저장하기
with open('output/word.txt', 'w', encoding='utf-8') as f:
    words =['sky', 'cloud', 'rain', 'sun', 'wind',
            'tree', 'flower', 'mountain', 'river', 'ocean']
    for word in words:
        f.write(f"{word} ")
# 파일 닫기는 with 블록이 끝나면 자동으로 처리됩니다.

# 파일 열기 - 읽기 모드
try:
    with open('output/word.txt', 'r', encoding='utf-8') as f:
        # content = f.read()
        # print(content)
        word = f.read().split() # 공백을 기준으로 단어 분리
        word = random.choice(word) # 랜덤으로 단어 선택
        print(word) # 랜덤으로 선택된 단어 출력
except FileNotFoundError as e:
    print("파일을 찾을 수 없습니다:", e)
```

영어 타자 연습 프로그램

- 영어 타자 게임

```
import random
import time

try:
  with open('output/word.txt', 'r', encoding='utf-8') as f:
      word = f.read().split() # 공백을 기준으로 단어 분리

  n = 1 #문제 번호
  print("[타자 게임] 준비되면 엔터를 누르세요")
  input() #엔터 입력 대기

  start = time.time() #게임 시작 시간 기록
```

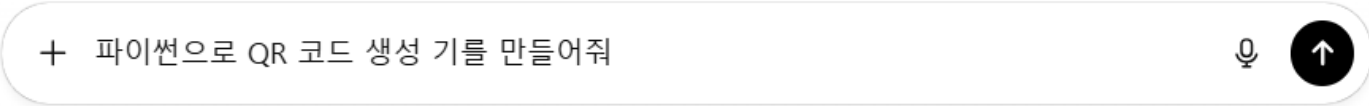
영어 타자 연습 프로그램

- 영어 타자 게임

```
    while n <= 10:
        print(f"\n문제 {n}")
        question = random.choice(word) #랜덤으로 단어 출력
        print(question)
        user = input() #사용자 입력 받기

        if question == user:
            print("통과!")
            n += 1 #문제 번호 증가
        else:
            print("오타! 다시 도전!")

    end = time.time() #게임 종료 시간 기록
    et = end - start #총 게임 시간 계산
    print(f"게임 시간: {et:.2f}초")
except FileNotFoundError:
    print("파일을 찾을 수 없습니다.")
```

QR 코드 생성기

- **챗GPT 프롬프트 입력**



- qrcode 모듈 설치하기

  pip install qrcode[PIL]

- qr 코드 스캔

  ☞ https://www.naver.com

QR 코드 생성기

- QR 코드

```
import qrcode

data = "http://www.naver.com"

# QR 코드 생성
img = qrcode.make(data)

# QR 코드 이미지 저장
img.save("output/naver_qr.png")
print("QR 코드가 생성되었습니다.")

# QR 코드 이미지 보여주기
img.show()
```

AI 가계부 분석기

◆ AI 가계부 분석기

```
[지출 내역]
         date category  amount  memo
0  2026-04-01       식비   12000    점심
1  2026-04-01       교통    2500   지하철
2  2026-04-02       카페    5500    커피
3  2026-04-03       식비   18000    저녁
4  2026-04-04       쇼핑   43000  생활용품

[카테고리별 합계]
category
쇼핑    43000
식비    30000
카페     5500
교통     2500
Name: amount, dtype: int64

총 지출: 81,000원
{'교통': 2500, '쇼핑': 43000, '식비': 30000, '카페': 5500}

[AI 소비 분석 리포트]
1. 쇼핑 지출이 큽니다. 필요/충동 구매를 구분해보세요.

[절약 목표 제안]
다음 달 목표 지출: 72,900원 (10% 절감)
```

AI 가계부 분석기

◆ AI 가계부 분석기

```python
import pandas as pd

# 샘플 데이터 생성 또는 CSV 불러오기
# CSV 형식: date,category,amount,memo
# 예: 2026-04-01,식비,12000,점심

FILE = 'expenses.csv'

try:
    df = pd.read_csv(FILE)
except FileNotFoundError:
    sample = [
        ['2026-04-01','식비',12000,'점심'],
        ['2026-04-01','교통',2500,'지하철'],
        ['2026-04-02','카페',15500,'커피'],
        ['2026-04-03','식비',18000,'저녁'],
        ['2026-04-04','쇼핑',43000,'생활용품'],
    ]
    df = pd.DataFrame(sample, columns=['date','category','amount','memo'])
```

AI 가계부 분석기

◆ AI 가계부 분석기

```
print('\n[지출 내역]')
print(df)

print('\n[카테고리별 합계]')
print(df.groupby('category')['amount'].sum().sort_values(ascending=False))

monthly_total = df['amount'].sum()
print(f'\n총 지출: {monthly_total:,}원')

# 간단한 AI 스타일 분석 규칙
advice = []
category_sum = df.groupby('category')['amount'].sum().to_dict()
# print(category_sum) #{'교통': 2500, '쇼핑': 43000, '식비': 30000, '카페': 15500}
```

AI 가계부 분석기

◆ AI 가계부 분석기

```
print(category_sum.get('카페',0)) # 카페 지출이 없을 때 0 반환
if category_sum.get('카페',0) > 10000:
    advice.append('카페 지출이 높습니다. 텀블러 사용이나 횟수 조절을 추천합니다.')
if category_sum.get('식비',0) > monthly_total * 0.4:
    advice.append('식비 비중이 높습니다. 주간 예산을 설정해보세요.')
if category_sum.get('쇼핑',0) > 30000:
    advice.append('쇼핑 지출이 큽니다. 필요/충동 구매를 구분해보세요.')
if not advice:
    advice.append('지출 패턴이 안정적입니다. 현재 소비 습관을 유지하세요.')
```

AI 가계부 분석기

◆ AI 가계부 분석기

```
print('\n[AI 소비 분석 리포트]')
# 1부터 시작하는 번호와 함께 조언 출력
for i, a in enumerate(advice,1):
    print(f'{i}. {a}')

print('\n[절약 목표 제안]')
print(f'다음 달 목표 지출: {int(monthly_total*0.9):,}원 (10% 절감)')


# csv 파일로 저장
df.to_csv(FILE, index=False)
```

AI 가계부 분석기

◆ AI 가계부 분석기

```python
# enumerate()로 리스트 항목과 인덱스 출력
colors = ['빨강', '초록', '파랑']
print('\n색상 목록:')
for index, color in enumerate(colors):
    print(f'{index}: {color}')

# enumerate() 예제
fruits = ['사과', '바나나', '체리']
print('과일 목록:')
for index, fruit in enumerate(fruits, start=1):
    print(f'{index}. {fruit}')

# enumerate()를 사용하여 딕셔너리 항목 출력
person = {'이름': '홍길동', '나이': 30, '직업': '개발자'}
print('\n개인 정보:')
for index, (key, value) in enumerate(person.items(), start=1):
    print(f'{index}. {key}: {value}')
```

AI 가계부 분석기

◆ AI 가계부 분석기

```python
import pandas as pd

# 천단위 구분 쉼표와 "원"을 붙이는 함수
def show_amount(x):
    return f"{x:,.0f}원"

df = pd.read_csv("expenses.csv")
print(df)

print('\n[지출 내역]')
print(df['amount'].apply(show_amount))

print('\n[카테고리별 합계]')
category_amount = df.groupby('category')['amount'].sum()
print(category_amount.apply(show_amount).sort_values(ascending=False))
```

```
[지출 내역]
0    12,000원
1     2,500원
2    15,500원
3    18,000원
4    43,000원
```

json(javascript object notation)

- **json 데이터**

JSON (JavaScript Object Notation) 은

→ 데이터를 **문자열 형태로 표현하는 규칙(포맷)** 입니다.

- **기본 구조**

JSON은 **키(key)와 값(value)** 으로 이루어져 있습니다.

```
{
  "name":  " 이순신 " ,
   " age " : 45,
  "isStudent": false
}
```

"name" → key (항상 문자열)

"홍길동" → value

json(javascript object notation)

- **리스트**

```
{
  "fruits": ["사과", "바나나", "포도"]
}
```

- **객체**

```
{
  “product”: {
    “name”: “마우스",
    “price": 30000
  }
}
```

- **사용되는 곳**
  - API 응답 데이터 형식
  - 설정 파일 (config)
  - 서버 ↔ 클라이언트 데이터 통신

json(javascript object notation)

- **json 데이터**

```
제품 목록
--------------------------------------------------
ID: 1
Name: 무선 마우스
Price: 32000
Description: 무선 마우스에 대한 설명입니다.
Image: mouse.png
--------------------------------------------------
ID: 2
Name: 무선 키보드
Price: 45000
Description: 무선 키보드에 대한 설명입니다.
Image: keyboard.png
--------------------------------------------------
ID: 3
Name: USB 메모리
Price: 15000
Description: USB 메모리에 대한 설명입니다.
Image: usb.png
--------------------------------------------------
```

json(javascript object notation)

- **json 데이터**

```python
# json 파일 읽기
import json
import os

# 현재 파일의 디렉토리 경로를 가져옵니다.
base_dir = os.path.dirname(os.path.abspath(__file__))

# json 파일 경로를 설정합니다.
file = os.path.join(base_dir, 'products.json')

# json 파일을 읽어서 데이터를 출력합니다.
with open(file, 'r', encoding='utf-8') as f:
    data = json.load(f)
print(data)
```

json(javascript object notation)

- **json 데이터**

```python
# 제품 정보 출력
print("\n첫 번째 제품 정보")
print("-" * 50)
print(f"ID: {data[0]['id']}")
print(f"Name: {data[0]['name']}")
print(f"Price: {data[0]['price']}")
print(f"Description: {data[0]['description']}")
print(f"Image: {data[0]['image']}")

print("\n제품 목록")
print("-" * 50)
for product in data:
    print(f"ID: {product['id']}")
    print(f"Name: {product['name']}")
    print(f"Price: {product['price']}")
    print(f"Description: {product['description']}")
    print(f"Image: {product['image']}")
    print("-" * 50)
```

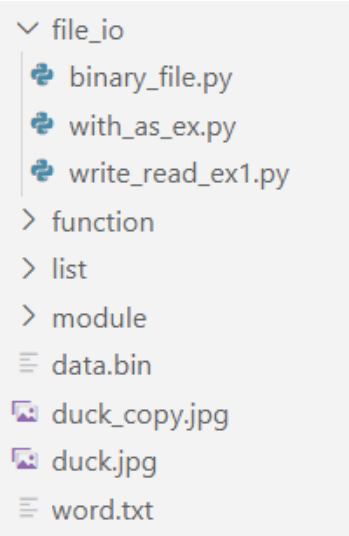
바이너리 파일 읽고 쓰기

❖ 바이너리 파일

바이너리 파일이란 화상, 음성 등의 대부분의 파일로 0과 1로 이루어진 파일이다.

open("파일 위치", 'wb')

모드	설명
wb	쓰기
rb	읽기



바이너리 파일 읽고 쓰기

❖ 바이너리 파일

```python
# 바이너리 파일 읽기/쓰기 예제
with open("data.bin", "wb") as f:
    # 정수 데이터를 바이너리 형식으로 쓰기
    for i in range(1, 6):
        # 4바이트 정수로 변환하여 쓰기
        f.write(i.to_bytes(4, byteorder='little'))

with open("data.bin", "rb") as f:
    print("바이너리 파일에서 읽은 정수들:")
    while True:
        bytes_read = f.read(4)
        if not bytes_read:
            break
        # 4바이트 정수를 다시 정수로 변환
        number = int.from_bytes(bytes_read, byteorder='little')
        print(number)
```

```
바이너리 파일에서 읽은 정수들:
1
2
3
4
5
```

바이너리 파일 읽고 쓰기

❖ 이미지 복사하기

이미지 파일 읽어와서 다른 이름으로 쓰기

```
# 이미지 파일 읽기
with open("duck.jpg", "rb") as f1:
    data = f1.read()

# 이미지 파일 쓰기
with open(".duck_copy.jpg", "wb") as f2:
    f2.write(data)
```

pickle 모듈

- **pickle** 모듈
  - 객체의 형태를 그대로 유지하면서 파일에 저장하고 불러올 수 있는 모듈이다.
  - 이때 객체란, 리스트나 딕셔너리등의 자료구조도 포함한다.

모드	설명
pickle.dump	쓰기
pickle.load	읽기

pickle 모듈

- **객체 자료 쓰고 읽기**

```python
import pickle

# 딕셔너리를 파일에 저장합니다.
try:
    with open('./output/data.txt', 'wb') as f:
        dic = {1: '강아지', 2: '고양이', 3: '닭'}
        pickle.dump(dic, f)
except Exception as e:
    print(f"예외 발생: {e}")


# 파일에서 딕셔너리를 읽어옵니다.
try:
    with open('./output/data.txt', 'rb') as f:
        data = pickle.load(f)
        print(data)
except Exception as e:
    print(f"예외 발생: {e}")
```

data.txt

```
<ENQ><)NULNULNULNULNULNULNUL}<(K<SOH< 강아지<K<STX<  고양이<K<ETX<<ETX닭<u.
```